GrovBase Shot – jak działa automatyzacja

Nov 6, 2026 · @Kuba

Cel i zakres

Automatyzacja zamienia zdjęcia produktu i opis z Allegro w gotowy zestaw 1–10 zdjęć lifestyle do galerii aukcji, w siedmiu krokach bez udziału człowieka między formularzem a galerią. Silnik robi to, co w testach robiłem ręcznie: kartę produktu, plan ujęć, prompty i generację w Nano Banana Pro. Reguły bierze z dokumentu „GrovBase Shot – specyfikacja silnika lifestyle".

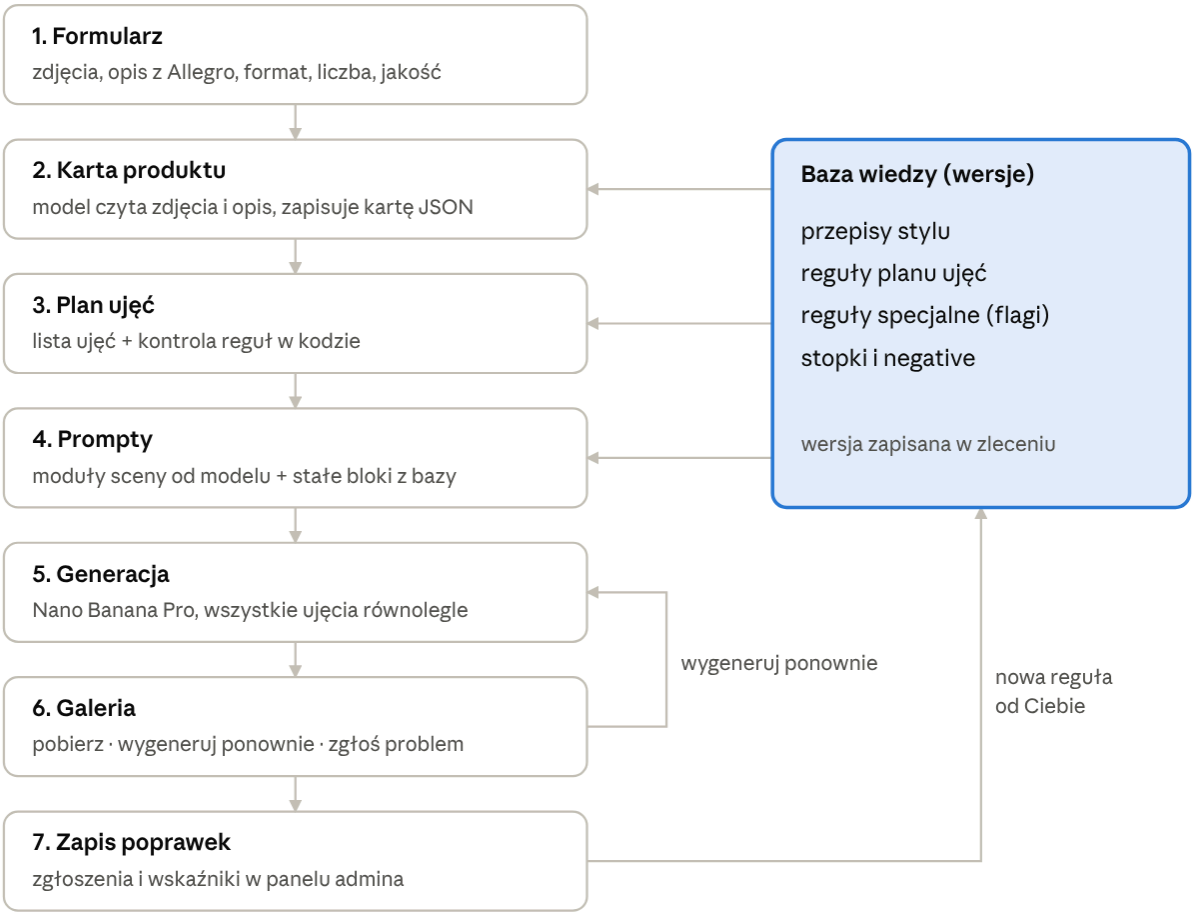
W automatyzacji nie ma:

- sprawdzania jakości zdjęć od klienta i odrzucania ich (ostrość, zgodność, ucinanie) – każde przesłane zdjęcie idzie dalej;
- automatycznej oceny wygenerowanych zdjęć i odrzucania ich przez model – klient dostaje wszystko, co wygenerowano, i sam decyduje o ponownej generacji;
- trybu reklamowego z napisami (osobny etap później).

Model wizyjny nadal **czyta** zdjęcia w kroku 2, ale tylko po to, żeby opisać produkt i nazwać rolę każdego zdjęcia (całość, detal, użycie). Niczego nie odrzuca. Bez tego plan nie wie, które zdjęcie dołączyć do którego ujęcia.

Przebieg w skrócie

Od formularza do galerii bez człowieka; poprawki wracają przez bazę wiedzy



przebieg zlecenia · 7 kroków, pętla ponownej generacji, pętla poprawek

Klient dotyka procesu tylko w krokach 1 i 6. Poprawki trafiają do silnika wyłącznie przez nową wersję bazy wiedzy, którą zatwierdzasz Ty.

Krok 1: formularz wejściowy

Klient wypełnia jeden formularz, a aplikacja tworzy z niego zlecenie i rezerwuje kredyty. Sprawdzenie jest wyłącznie techniczne: typ pliku, rozmiar, liczba zdjęć, niepuste pola.

Pole	Wartości	Uwagi
Zdjęcia produktu	1–10 plików JPG, PNG lub WEBP	kolejność zachowana; każde dostaje id (Z1, Z2…)
Opis z Allegro	tekst wklejony w całości	tytuł + opis + specyfikacja
Scenariusz	lifestyle	„reklamowa” ukryta do czasu testu

Pole	Wartości	Uwagi
Format	1:1, 4:5, 3:4, 16:9, 9:16	jeden dla całego zestawu
Liczba zdjęć	1–10	wpływa na rozkład ujęć w kroku 3
Jakość	1K, 2K, 4K	przekazywana do API generacji
Uwagi (opcjonalne)	krótki tekst	np. „bez dzieci”, „ogród zamiast kuchni”

Po kliknięciu „Generuj”:

1. Pliki trafiają do magazynu (np. S3), a w bazie powstaje zlecenie ze statusem `nowe`.
2. Aplikacja rezerwuje kredyty: liczba zdjęć × cena za zdjęcie. Pobranie następuje dopiero po dostarczeniu, a niedostarczone zdjęcia wracają na konto.
3. Zlecenie trafia do kolejki zadań w tle (np. BullMQ, Celery albo Cloud Tasks). Cały proces trwa kilka minut, więc nie może działać w jednym zapytaniu HTTP.
4. Klient widzi ekran postępu ze statusem zlecenia (krok 2 → 3 → 4 → 5).

Krok 2: karta produktu

Jedno wywołanie modelu tekstowo-wizyjnego zamienia zdjęcia i opis w kartę produktu w formacie JSON. Wszystkie kolejne kroki czytają kartę, a nie zdjęcia i opis bezpośrednio.

Wejście wywołania: wszystkie zdjęcia z ich id, opis z Allegro, uwagi klienta. W instrukcji systemowej: lista pól karty, lista przepisów stylu z kategoriami, zasady uczciwości i procedura 8 punktów ze specyfikacji.

Wyjście (JSON):

Pole	Co zawiera	Przykład (lampa do roślin)
produkt	czym jest, wariant	lampa do uprawy, 3 głowice
wymiary	z opisu albo założenie oznaczone jako założenie	listwa 25 cm, całość do 40 cm
skala	porównania z przedmiotami znanymi modelowi	listwa jak krótszy bok kartki A4
budowa	elementy w kolejności, kolor, materiał, liczby	klips → 3 tulejki → 3 ramiona → 3 listwy
napisy	co jest na produkcie i ma zostać	3H, 9H, 12H na pilocie

Pole	Co zawiera	Przykład (lampa do roślin)
stany	stany produktu i zdjęcia, które je pokazują	świeci / wyłączona
zdjęcia	dla każdego id: rola (całość, detal, użycie, opakowanie), widok, stan, grafiki do zakazu kopiowania	Z4: użycie, linie wymiarowe i gwiazdki
argumenty	argumenty sprzedażowe z opisu, po jednym na ujęcie	timer, 3 tryby, klips
zastosowania	miejsca użycia z opisu	parapet, biurko, blat
grupa docelowa	kto kupuje i używa	miłośnicy roślin 25–65 lat
uczciwość	ograniczenia	IP20: bez wody; tylko czerwone i niebieskie diody
sezon	produktu i użycia	całoroczny, używany jesienią i zimą
ryzyka	co model typowo psuje	dokłada ramiona, białe światło
flagi kategorii	cechy, które włączają reguły specjalne	`swieci`, `miekki`, `sezonowy`, `dla_zwierzat`
przepis stylu	id przepisu z bazy wiedzy	`domowa_zielen`

Kod sprawdza JSON względem schematu. Gdy brakuje pola albo id zdjęcia nie istnieje, wywołanie jest powtarzane raz z komunikatem o błędzie.

Zdjęcie z grafikami (wymiary, strzałki, inne produkty) nie jest odrzucane. Karta zapisuje, czego nie wolno z niego kopiować, a krok 4 dopisuje to do promptu.

Krok 3: plan ujęć

Drugie wywołanie modelu dostaje kartę i liczbę zdjęć i zwraca listę ujęć w kolejności galerii. Tu silnik decyduje, co pokazuje każde zdjęcie; od tego najbardziej zależy jakość zestawu.

Wejście: karta (JSON), liczba zdjęć, format, uwagi klienta. W instrukcji systemowej: reguły planu ze specyfikacji (treść, kadr i różnorodność, wierność, ludzie, pora roku), tabela kierunków kreatywnych, kontrola anty-stock i wybrany przepis stylu.

Wyjście (JSON), jedno ujęcie: numer, argument sprzedażowy, typ (hero, użycie, detal), kierunek kreatywny, miejsce, światło otoczenia, typ kadru, ludzie (brak, dłonie, postacie), referencje (id zdjęć w kolejności dołączania) i jedno zdanie opisu sceny.

Część reguł kod sprawdza mechanicznie, bez modelu:

- ten sam typ kadru najwyżej 2 razy;
- złota godzina najwyżej w 2 ujęciach;
- każde zdjęcie z rolą „detal” ma własne ujęcie;
- ujęcia użycia i detali mniej więcej po połowie;
- liczba ujęć z postaciami i dłońmi zgodna z tabelą „Ludzie i zwierzęta” (np. przy 6–10 zdjęciach 2–3 postacie, 1–2 dłonie);
- referencje istnieją i mają ten sam stan produktu co scena;
- w ujęciu z całym produktem nie ma referencji z samym fragmentem.

Naruszenie = jedno ponowne wywołanie z listą błędów („ujęcia 3 i 7 mają ten sam kadr”). Jeśli i ono się nie uda, plan idzie dalej, a naruszenie zapisuje się w logu do wglądu.

Krok 4: składanie promptów

Prompt każdego ujęcia składa kod z modułów w stałej kolejności. Model pisze tylko części zależne od produktu i sceny, a stałe teksty pochodzą z bazy wiedzy. Dzięki temu sprawdzone sformułowania (stopki, dawka światła, zakazy) nie zmieniają się między zleceniami.

Moduł	Kto go tworzy	Ile razy
KIERUNEK	baza wiedzy: blok wybranego przepisu stylu	raz na zlecenie
PRODUKT	model, z karty: budowa, kolor, napisy, czego produkt nie jest, czego nie kopiować z referencji	raz na zlecenie, identyczny we wszystkich ujęciach z całym produktem
SKALA	model, z karty: wymiary + porównania	raz na zlecenie
Reguły specjalne	baza wiedzy, włączane flagami karty (np. `swieci` → blok dawki światła)	raz na zlecenie
STAN / CHWYT, MOMENT, KADR, POSTAĆ, SCENOGRAFIA	model, osobno dla każdego ujęcia z planu	na ujęcie

Moduł	Kto go tworzy	Ile razy
STOPKA	baza wiedzy, dobierana kodem po typie ujęcia (wnętrze, ludzie, detal)	na ujęcie
NEGATIVE	kod: baza wspólna + ryzyka z karty + dopiski ujęcia	na ujęcie

Wywołania są dwa: pierwsze tworzy bloki PRODUKT i SKALA, drugie wszystkie moduły scen naraz. Jedno wywołanie na wszystkie sceny pilnuje, żeby miejsca i światło się nie powtarzały.

Ujęcia detali dostają krótszy wzorzec: KIERUNEK, opis detalu skopiowanego 1:1 z referencji (zmienione tylko tło, światło i jeden element), KADR z kątem z referencji, STOPKA detalu, NEGATIVE.

Gotowy prompt każdego ujęcia zapisuje się w bazie razem z wersją bazy wiedzy, z której powstał. Bez tego nie da się później ustalić, która reguła dała zły wynik.

Krok 5: generacja obrazów

Każde ujęcie to osobne zadanie w kolejce, wysyłane do Nano Banana Pro równolegle z pozostałymi. Zestaw 10 zdjęć generuje się więc mniej więcej w czasie jednego obrazu, a nie dziesięciu.

Jedno zapytanie zawiera:

- prompt z kroku 4;
- zdjęcia referencyjne w kolejności z planu (prompt odwołuje się do nich jako „zdjęcie 1”, „zdjęcie 2”, więc kolejność musi się zgadzać);
- format i rozdzielczość z formularza.

Obsługa błędów technicznych (nie jakości):

Sytuacja	Reakcja
przekroczony czas, błąd 5xx, limit zapytań	2 ponowienia z odstępem rosnącym (np. 10 s, 30 s)
odmowa z powodów bezpieczeństwa	1 ponowienie z tym samym promptem, potem ujęcie oznaczone jako nieudane
ujęcie nieudane po ponowieniach	kredyty za nie wracają, klient widzi informację przy tym miejscu w galerii

Wynik trafia do magazynu, a ujęcie dostaje status `gotowe`. Gdy wszystkie ujęcia mają status końcowy, zlecenie przechodzi do galerii, a klient dostaje powiadomienie.

Każde ujęcie ma jedną generację. Bez automatycznej oceny nie ma kto wybrać lepszego z dwóch kandydatów, więc drugi kandydat byłby tylko kosztem. Zamiast tego klient ma przycisk ponownej generacji (krok 6).

Krok 6: galeria, wybór i ponowna generacja

Klient widzi zestaw w kolejności z planu, czyli tej, w jakiej zdjęcia mają stanąć na aukcji. Przy każdym zdjęciu są trzy akcje:

1. **Pobierz.** Pojedynczo albo cały zestaw jako ZIP z nazwami plików w kolejności galerii (01_hero.png, 02_klips.png…).
2. **Wygeneruj ponownie.** Ten sam prompt i te same referencje, nowa generacja. W testach to naprawiało błędy losowe: uciętą rękę, sklonowaną osobę, drugą lampę w tle. Nowa wersja nie kasuje poprzedniej, klient wybiera lepszą.
3. **Zgłoś problem.** Klient wybiera powód z listy i opcjonalnie dopisuje zdanie. Ten krok nie zmienia zdjęcia, tylko zasila krok 7.

Powody do wyboru: produkt zmieniony, zła skala, dłonie lub twarze, światło lub klimat, scena nie pasuje, inne.

W drugiej wersji można dodać **„Popraw z uwagą”**. Klient wpisuje, co zmienić („jaśniej”, „bez dziecka”), model przepisuje moduły sceny tego ujęcia (krok 4), a kod generuje je od nowa. To odpowiednik naszych poprawek przy błędach systematycznych, których sama ponowna generacja nie naprawi.

Gotowych zdjęć nie edytujemy. Każda zmiana to pełna generacja od zera, bo edycja psuje kolory.

Do decyzji: ile ponownych generacji jest w cenie. Propozycja: 1 bezpłatna na każde ujęcie, kolejne za kredyty. Bez automatycznej oceny klient zobaczy więcej błędów niż w pierwotnym projekcie, więc darmowa ponowna generacja chroni zaufanie.

Krok 7: zapis poprawek i nauka

Silnik nie uczy się sam. Uczy się tak jak w naszych testach: człowiek ogląda błędy, zapisuje regułę, a kolejne zlecenia korzystają z nowej wersji bazy wiedzy. Automatyczne dopisywanie reguł z uwag klientów szybko zebrałoby sprzeczne poprawki, jak „za ciemno” i „za jasno” przy tym samym przepisie.

Co zapisuje się przy każdym zleceniu: karta, plan, prompty, wersja bazy wiedzy, wszystkie wygenerowane obrazy, ponowne generacje i zgłoszenia z powodami.

Panel administratora (dla Ciebie) pokazuje:

- **wskaźnik ponownych generacji** według przepisu stylu i typu ujęcia – główny sygnał, że przepis nie działa;
- **zgłoszenia** pogrupowane po powodzie, z obrazem, promptem i kartą obok siebie;
- **porównanie wersji** – czy po zmianie reguły wskaźnik spadł.

Poprawka przebiega tak:

1. Widzisz powtarzający się błąd (np. trzy lampy ze słabym światłem).
2. Aktualizujemy specyfikację, a Ty wgrywasz ją w panelu jako nowy plik z jednym zdaniem opisu zmiany (sekcja „Wgrywanie nowej wersji wiedzy").
3. Powstaje nowa wersja bazy wiedzy, a kolejne zlecenia jej używają.
4. Opcjonalnie uruchamiasz zestaw testowy (ok. 10 produktów z różnych kategorii) na nowej wersji i porównujesz wyniki ze starą, zanim włączysz ją dla klientów.

Baza wiedzy, dane, koszty i kolejność budowy

Baza wiedzy silnika

Specyfikacja silnika zamienia się w pliki konfiguracyjne z numerem wersji. Kroki 2–4 wklejają je do instrukcji modelu albo bezpośrednio do promptu.

Element	Zawartość	Używa go
Przepisy stylu	na przepis: id, kategorie, blok KIERUNEK, zasady światła i scenografii, dopiski do negative	krok 2 (wybór), krok 4 (tekst)
Reguły planu	treść, kadr, wierność, ludzie, pora roku, kierunki kreatywne, anty-stock	krok 3
Reguły specjalne	bloki włączane flagą: produkty świecące, miękkie, sezonowe, dla zwierząt, kotwice skali	krok 4
Stopki	tekst na typ ujęcia	krok 4
Negative bazowy	lista wspólna dla wszystkich ujęć	krok 4

Wgrywanie nowej wersji wiedzy

Po każdej serii testów na nowych produktach aktualizuję specyfikację silnika, a Ty wgrywasz ją w panelu administratora jako nowy plik. Silnik zaczyna z niej korzystać bez zmian w kodzie.

Format pliku. Specyfikacja wyeksportowana z dokumentu jako Markdown (.md), ze stałymi nagłówkami: Przepisy stylu, Reguły planu, Reguły specjalne, Stopki, Negative bazowy. Kod wycina po nagłówkach teksty wstawiane do promptu dosłownie (bloki KIERUNEK, stopki, negative), a resztę pliku wkleja do instrukcji modeli w krokach 2–4. Dziś plik ma ok. 36 tys. znaków, czyli ok. 10 tys. tokenów. Mieści się w instrukcji systemowej, a buforowanie promptu u dostawcy modelu obniża koszt jego powtarzania.

Co dzieje się po wgraniu:

1. **Sprawdzenie struktury.** Kod sprawdza, czy są wszystkie wymagane nagłówki i czy każdy przepis ma blok KIERUNEK. Brak = komunikat z nazwą brakującej sekcji, plik nie zostaje przyjęty.
2. **Porównanie z aktywną wersją.** Panel pokazuje różnice: dodane, zmienione i usunięte reguły oraz przepisy.
3. **Test (opcjonalny).** Przycisk „Uruchom zestaw testowy” generuje ok. 10 produktów testowych na nowej wersji. Wyniki oglądasz obok wyników poprzedniej wersji.
4. **Aktywacja.** Klikasz „Włącz” i nowe zlecenia używają nowej wersji. Zlecenia w toku kończą na wersji, z którą wystartowały.
5. **Wycofanie.** Wszystkie poprzednie wersje zostają w bazie. Jeśli wskaźnik ponownych generacji wzrośnie, jednym kliknięciem wracasz do poprzedniej.

Każda wersja ma numer, datę, autora i jedno zdanie opisu zmiany (np. „dawka światła dla produktów świecących”). Numer wersji zapisuje się przy każdym zleceniu, więc przy każdym złym zdjęciu wiadomo, która wiedza je wygenerowała.

Model danych

Tabela	Najważniejsze pola
zlecenia	id, klient, status, scenariusz, format, jakość, liczba zdjęć, uwagi, wersja bazy wiedzy, kredyty zarezerwowane i pobrane
zdjecia_wejsciowe	id, zlecenie, kolejność, plik, rola i opis z karty
karty	zlecenie, JSON karty, przepis stylu, flagi
ujecia	id, zlecenie, numer, JSON planu, prompt, referencje, status

Tabela	Najważniejsze pola
generacje	id, ujęcie, plik, czy ponowna, czy wybrana przez klienta, czas, koszt
zgloszenia	generacja, powód, komentarz
baza_wiedzy	wersja, pliki, opis zmiany, data

Statusy zlecenia: `nowe` → `karta` → `plan` → `prompty` → `generacja` → `gotowe` (albo `czesciowo_gotowe`, gdy część ujęć się nie udała).

Koszty

Nano Banana Pro kosztuje 0,134 USD za obraz 2K. Kroki 2–4 to cztery wywołania modelu tekstowego, rzędu centów na zlecenie. Cenę kredytów warto liczyć na 1,5–2 generacje na dostarczone zdjęcie, nie na jedną.

Zestaw	1 generacja na zdjęcie	1,5–2 generacje na zdjęcie
5 zdjęć 2K	0,67 USD	1,01–1,34 USD
10 zdjęć 2K	1,34 USD	2,01–2,68 USD

Cena za 4K i aktualne limity zapytań API do sprawdzenia w cenniku Google przed ustaleniem cennika kredytów.

Kolejność budowy

1. **Skrypt silnika bez panelu (Marcel).** Kroki 2–5 jako skrypt uruchamiany z linii poleceń na 10 produktach z naszych testów. Wyniki porównujemy z zestawami, które zaakceptowałeś ręcznie.
2. **Baza wiedzy w plikach.** Specyfikacja jako plik Markdown ze stałymi nagłówkami, wgrywanie nowych wersji w panelu, sprawdzenie struktury, aktywacja i wycofanie.
3. **Formularz, kolejka i galeria** z pobieraniem i ponowną generacją.
4. **Kredyty:** rezerwacja, pobranie po dostarczeniu, zwrot za nieudane ujęcia.
5. **Panel administratora** ze zgłoszeniami i wskaźnikiem ponownych generacji.
6. **„Popraw z uwagą”** i tryb reklamowy jako kolejne wersje.